W2 — JavaScript ve ES6 | Sınav Çalışma Dosyası

Kaynak: zeki-ozen/MobileAppDevelopment / W2
Kapsam: Depodaki `W2/README.md` ve `W2/topics/00–10` numaralı dersler.
Kullanım: Açıklama + örnek kod + beklenen sonuç + sınavda dikkat edilecek noktalar. Kodlar tarayıcı konsolunda veya Node.js ortamında denenebilir.

> **Önemli ayrım:** Bu hafta **JavaScript dili** anlatılır. Buradaki `console.log()` örnekleri Expo Snack'teki `App.js` ekranına yazı basmaz; mobil ekranda yazı göstermek için daha sonraki haftalarda `<Text>` kullanılır.

1. JavaScript nedir, nerede çalışır?

JavaScript; değişken, koşul, döngü, fonksiyon ve nesne yazmayı sağlayan programlama dilidir. Tarayıcıda HTML ile, Node.js ile sunucu/terminal ortamında, React Native'de uygulama mantığında kullanılır. **ES6**; `let`, `const`, ok fonksiyonları, modüller, destructuring, spread gibi modern sözdizimlerini yaygınlaştıran 2015 standart sürümüdür.

```
console.log('Merhaba JavaScript');
const ad = 'Ünal';
console.log(`Merhaba ${ad}`); // Merhaba Ünal
```

- **Tarayıcı:** `F12` → Console'a yazıp çalıştır.
- **Node.js:** `node deneme.js` komutuyla `.js` dosyasını çalıştır.
- **HTML içinde:** `<script src="app.js" defer></script>`; `defer` HTML ayrıştırması tamamlandıktan sonra script'in çalışmasını sağlar (klasik dış script).
- **Modül içinde:** `<script type="module" src="main.js"></script>`; modüller kendi kapsamına sahiptir, varsayılan olarak gecikmeli yürütülür.

```
<!doctype html>
<html lang="tr">
  <head><meta charset="UTF-8"><script src="app.js" defer></script></head>
  <body><h1>JavaScript denemesi</h1></body>
</html>
```

2. Değişkenler: `let`, `const`, `var`

İfade	Yeniden değer atanır mı?	Blok kapsamı	Tercih
`const`	Hayır	Evet	Değişmeyecek bağlar için ilk tercih
`let`	Evet	Evet	Sayaç/güncellenecek değer

İfade	Yeniden değer atanır mı?	Blok kapsamı	Tercih
`var`	Evet	Hayır (fonksiyon kapsamı)	Eski kodlarda görülebilir

```
const okul = 'Üniversite';
let puan = 40;
puan = 55;            // olur
// okul = 'Lise';    // TypeError: const'a yeniden atama yok

const ogrenci = { ad: 'Ünal' };
ogrenci.ad = 'Ali';   // olur: nesnenin İÇERİĞİ değişti
// ogrenci = {};    // olmaz: const bağını değiştirmeye çalışır
```

Hoisting / Temporal Dead Zone (TDZ): `var` tanımı yukarı taşınmış gibi görülür ve ilk değeri `undefined` olabilir; `let` ve `const` tanımlanmadan önce okunamaz. `const` tanımlanırken ilk değer zorunludur.

```
console.log(a); // undefined
var a = 5;
// console.log(b); // ReferenceError (TDZ)
let b = 8;
```

Sınav tuzağı: `const` ile tanımlanmış bir **diziye** `push()` yapılabilir; yasak olan dizinin değişkene tümden başka dizi olarak atanmasıdır.

3. Veri tipleri ve tip dönüşümü

Tip	Örnek	Not
`string`	`'Merhaba'`	Metin, template literal: `` `Merhaba ${ad}` ``
`number`	`12`, `3.5`, `NaN`	Tamsayı ve ondalık aynı tip
`boolean`	`true`, `false`	Koşullar
`undefined`	`let x;`	Değer henüz verilmemiş
`null`	`let x = null;`	Bilinçli boş değer
`bigint`	`123n`	Büyük tamsayı
`symbol`	`Symbol('id')`	Benzersiz simge
`object`	`{x:1}`, `[1,2]`	Başvuru türü; diziler teknik olarak object

```
console.log(typeof 12);             // 'number'
console.log(typeof '12');           // 'string'
console.log(typeof null);           // 'object' (dilsel tarihsel özellik)
console.log(Array.isArray([1,2]));  // true
console.log(Number('42'));          // 42
console.log(String(42));            // '42'
console.log(Boolean(''));           // false
```

`Number('abc')` sonucu `NaN`; `Number.isNaN(...)` ile kontrol edilir. `null` ile `undefined` farklıdır. Nesneler/diziler karşılaştırılırken **referans** dikkate alınır.

```
console.log([1] === [1]); // false, iki farklı dizi nesnesi
const x = [1]; const y = x;
console.log(x === y);     // true
```

4. Operatörler ve önemli farklar

Operatör	Amaç	Örnek ve sonuç
`+ - * / % **`	Aritmetik	`7 % 3` → `1`, `2 ** 3` → `8`
`=== / !==`	Tip dahil eşitlik	`5 === '5'` → `false`
`==`	Dönüştürerek eşitlik	`5 == '5'` → `true`; genelde kaçın
`&&`	VE, kısa devre	`true && false` → `false`
`||`	VEYA / ilk truthy	`'' || 'varsayılan'` → `'varsayılan'`
`??`	nullish varsayılan	`0 ?? 10` → `0`
`?.`	Güvenli özellik okuma	`kisi?.adres?.il`
`? :`	Üçlü koşul	`not >= 50 ? 'Geçti' : 'Kaldı'`

```
const giris = 0;
console.log(giris || 10); // 10; 0 falsy
console.log(giris ?? 10); // 0; null/undefined değil
console.log(3 + '4');     // '34' (string birleştirme)
console.log(3 - '1');     // 2 (sayısal dönüşüm)
```

Falsy değerler: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN`. `[]` ve `{}` **truthy** değerlerdir.

5. Koşullar, `switch`, guard clause

```
function notDurumu(not) {
  if (not < 0 || not > 100) return 'Geçersiz not'; // guard
  if (not >= 85) return 'AA';
  else if (not >= 50) return 'Geçti';
  return 'Kaldı';
}
console.log(notDurumu(72)); // Geçti
```

```
const gun = 2;
switch (gun) {
  case 1: console.log('Pazartesi'); break;
  case 2: console.log('Salı'); break;
  default: console.log('Bilinmeyen gün');
}
```

`break` olmazsa normal `switch` içinde sonraki `case` bloklarına da geçilir. `switch` eşleştirmesi katı eşitlik gibi davranır.

6. Döngüler: `for`, `while`, `for...of`, `forEach`

```
for (let i = 1; i <= 3; i++) console.log(i); // 1,2,3
let k = 0;
while (k < 2) { console.log(k); k++; }        // 0,1
const renkler = ['kırmızı','mavi'];
for (const renk of renkler) console.log(renk);
renkler.forEach((renk, index) => console.log(index, renk));
```

- `for...of`: Dizinin **değerlerini** gezer.
- `for...in`: Nesnenin **anahtarlarını** gezer; diziler için genelde `for...of` tercih edilir.
- `break`: Döngüyü bitirir. `continue`: O turu atlar.
- `forEach` içindeki `return`, dıştaki fonksiyondan veya döngüden çıkış gibi davranmaz. Erken durmak gerekiyorsa `for...of` veya `some` / `find` kullan.

7. Fonksiyonlar, arrow function, parametreler

```
function topla(a, b) { return a + b; }
const carp = (a, b) => a * b;
const selamla = (ad = 'Misafir') => `Selam ${ad}`;
console.log(topla(3,4));  // 7
console.log(carp(3,4));   // 12
console.log(selamla());   // Selam Misafir
```

```
const toplam = (...sayilar) => sayilar.reduce((t, s) => t + s, 0);
console.log(toplam(2, 3, 5)); // 10
const kare = x => x * x;       // tek parametrede parantez opsiyonel
const nesneUret = ad => ({ ad }); // nesne döndürürken parantez kullan
```

Dikkat: `() => { x: 1 }` nesne döndürmez; blok olarak değerlendirilir. Ok fonksiyonlarının kendi `this` bağlamı yoktur; dıştaki bağlamı kullanır.

8. Diziler: `map`, `filter`, `find`, `reduce`

```
const sayilar = [1, 2, 3, 4];
const ikiKati = sayilar.map(x => x * 2);         // [2,4,6,8]
const ciftler = sayilar.filter(x => x % 2 === 0); // [2,4]
const ilkBuyuk = sayilar.find(x => x > 2);        // 3
const toplam = sayilar.reduce((t, x) => t + x, 0); // 10
console.log(sayilar.includes(3));                // true
```

Metot	Ne üretir?	Orijinal diziyi değiştirir mi?
`map`	Her elemana karşılık yeni dizi	Hayır
`filter`	Koşulu sağlayanlardan yeni dizi	Hayır
`find`	İlk uygun öğe veya `undefined`	Hayır
`reduce`	Birikimli tek değer (veya yapı)	Hayır
`push`, `pop`, `splice`	Dizinin içeriğini günceller	**Evet**
`slice`	Kesit içeren yeni dizi	Hayır

```
const kisiler = [
  { id: 1, ad: 'Ünal', puan: 75 },
  { id: 2, ad: 'Ayşe', puan: 90 },
  { id: 3, ad: 'Can', puan: 35 },
];
const gecenAdlar = kisiler
  .filter(k => k.puan >= 50)
  .map(k => k.ad);
console.log(gecenAdlar); // ['Ünal','Ayşe']
```

9. Nesneler, destructuring, spread / rest

```
const kullanici = { id: 5, ad: 'Ünal', sehir: 'İstanbul' };
const { ad, sehir } = kullanici;
const liste = [10, 20, 30];
const [birinci, ...kalan] = liste;
const guncel = { ...kullanici, sehir: 'Ankara' };
const eklenmis = [...liste, 40];
```

`...` iki bağlamda kullanılır: **spread** (dizi/nesneyi yaymak) ve **rest** (kalanları toplamak). Nesne spread işlemi **sığ kopyadır**; içteki nesneler aynı referansı paylaşabilir.

10. Modüller: `export` ve `import`

mathUtils.js dosyası:

```
export const PI = 3.14;
export function topla(a, b) { return a + b; }
export default function kare(x) { return x * x; }
```

main.js dosyası:

```
import kare, { PI, topla } from './mathUtils.js';
console.log(topla(2,3), kare(4), PI); // 5 16 3.14
```

- **Named export:** İçe alırken `{ }` kullanılır, ad eşleşir (`as` ile takma ad verilebilir).
- **Default export:** İçe alırken `{ }` kullanılmaz; dosya başına en fazla bir default export vardır.
- Tarayıcıda modül kullanımında `type="module"` gerekir; yerel dosya (`file://`) yerine yerel sunucu daha güvenilirdir.

11. Mini alıştırma — Not listesi (çalışır JS)

Aşağıdaki kod doğrudan Node.js/tarayıcı konsolunda çalışır. `filter` + `map` + `reduce` aynı anda kullanılır:

```
const notlar = [40, 75, 90, 45, 60];
const gecenler = notlar.filter(n => n >= 50);
const etiketler = gecenler.map(n => `Not: ${n}`);
const ortalama = notlar.reduce((t, n) => t + n, 0) / notlar.length;
console.log(gecenler);   // [75,90,60]
console.log(etiketler);  // ['Not: 75','Not: 90','Not: 60']
console.log(ortalama);   // 62
```

12. Sınav tarzı sorular — cevaplarıyla

S1. `const x = [2, 3]; x.push(4);` hata verir mi? **Cevap:** Hayır; değişkenin işaret ettiği dizi değiştirilir.

S2. `console.log(0 || 15, 0 ?? 15)` ne yazar? **Cevap:** `15 0`.

S3. `['a','b','c'].map((v,i) => i + v)` sonucu? **Cevap:** `['0a','1b','2c']`.

S4. `5 == '5'` ve `5 === '5'` sonucu? **Cevap:** `true`, `false`.

S5. Bir listeden yalnızca 50 üstü notları almak için hangi metot? **Cevap:** `filter(n => n > 50)`.

S6. `const { ad } = kullanici` ne yapar? **Cevap:** `kullanici.ad` özelliğini `ad` adlı yerel değişkene ayırır.

S7. `for...of` ile `for...in` farkı? **Cevap:** İlki değer, ikincisi özellik anahtarı üretir.

S8. `import { topla }` ve `import topla` farkı? **Cevap:** İlki named, ikincisi default export içindir.

13. Sınavda hızlı bakış

İhtiyaç	Hazır kalıp
Değişken	`const ad='Ünal'; let sayac=0;`
Koşul	`const yazi = n>=50 ? 'Geçti' : 'Kaldı';`
Döngü	`for (const x of dizi) { console.log(x); }`
Diziye yeni öğe (değiştirmeden)	`const yeni = [...eski, eleman];`
Elemanı sil (yeni dizi)	`const yeni = dizi.filter(x => x.id !== id);`
Özellik güncelle	`const yeni = {...kisi, ad:'Ali'};`
Sayı toplamı	`dizi.reduce((t,x) => t+x, 0)`
Varsayılan değer	`deger ?? varsayilan`

Kaynak dosyalar: W2/README.md; W2/topics.
Bu çalışma rehberi ders deposunun eğitim konularına göre özgün Türkçe özet olarak hazırlanmıştır; hocanın birebir sınav soruları değildir.